



FADO — Training Speed

Removing a redundant backward pass, and where the time actually goes

Tiago Silva

CISUC · Data-Centric AI Lab · 17 July 2026

Department of Informatics Engineering, University of Coimbra

The change: an analytic fairness gradient

Each online step ran **two** backward passes: one for the task loss, and a second (on a persistent tape, once per sample) only to read $\partial n / \partial W$, $\partial n / \partial b$ for the fairness penalty.

Why the second pass was never needed.

The fairness penalty is a function of the *node activations*, so its gradient is the node's own **local** Jacobian — there is no upstream chain to assemble. With $n = \sigma(z)$, $z = (W^\top x + b) / \tau$:

$$\frac{\partial n}{\partial W} = x \sigma'(z)^\top / \tau, \quad \frac{\partial n}{\partial b} = \sigma'(z) / \tau, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

And $\sigma(z)$ is the forward pass's `node_decisions` — already computed. One elementwise product, one outer product, per tree. No tape.

- The **task loss keeps autodiff**: its target is the prediction, so it genuinely needs the full upstream chain (leaf mixture, gates, θ).
- Applied to all three training loops · the tape is no longer persistent · 'leaf' mode still uses autodiff.

Autodiff is worth its cost only when you need the *upstream* chain. The fairness term never leaves the top of the graph.

Verified: bit-identical

Unit: both paths, same state

`gradient_w`, `gradient_b`, `agg_y` all differ by **0.00e+00**; counts match.

Across $\tau \in \{1.0, 0.35, 0.1\}$ and both update rules (vanilla, EMA).

End-to-end A/B

Original code restored and re-run on the same seed, 400 online steps.

DP, EO and accuracy trajectories **and** all 12 final weight tensors: **identical**.

Why exactly zero, and not just close?

Because the analytic path **reuses the forward's** $\sigma(z)$ instead of recomputing z , there is not even float32 rounding drift. (Recomputing z gives $\sim 1\text{e-}6$; reusing it gives 0.)

Consequence: **the published numbers cannot move**. This is a pure speed change.

Bonus: trees are now indexed directly instead of inferred from gradient shapes — immune to a θ whose leading dimension equals `data_dim`.

Measured: the honest result

The replaced pass: $\sim 4.5\text{ ms} \rightarrow \sim 8\text{ }\mu\text{s}$ per sample.

End-to-end: $24.44\text{ s} \rightarrow 22.72\text{ s} = 1.08\times$.

The arithmetic is consistent: $\sim 61\text{ ms}$ per step, and we removed $\sim 4.5\text{ ms} \approx 7\%$.

The fairness backward was never the bulk of the step.

Profile (train_online, 400 steps)	cum
compute_fairness_gradients	11.84 s
convert_to_tensor (195,565 calls)	8.13 s
_gradient_function (task loss)	3.91 s
accumulate_fairness_stats (target)	0.57 s
total	26.28 s

The change did exactly its job — `accumulate_fairness_stats` fell off the profile. But the bottleneck was somewhere else entirely: `compute_fairness_gradients` is **45% of the runtime**.

Next: the actual bottleneck

`compute_fairness_gradients` operates on state that is **already NumPy** (`gradient_w`, `gradient_b` are plain arrays) — but rebuilds every term with Python `sum()` comprehensions and per-element `tf.convert_to_tensor`, once per gradient tensor, every step.

≈**196,000 tensor constructions per 400 steps** — to do arithmetic on arrays of a few dozen numbers.

Fix: rewrite it in pure NumPy. No TF in the loop, no wrapping, no host syncs. Same treatment as today: prove bit-identical, then A/B the full trajectory.

- Then: **incremental fairness window** — $O(1)$ running counters instead of recomputing DP/EO over the whole window every step.
- Then: **graph the forward** with `@tf.function` (eager today) · batch the trees instead of a Python loop.

Lesson worth keeping: the derivation was right and the component got $540\times$ faster — but it was 7% of the step. **Profile first, then optimise.**

Summary

Done. The redundant fairness backward pass is gone from all three training loops — proven bit-identical (state, trajectories and final weights), so no result changes.

Honest gain. $1.08\times$ end-to-end. The component is $\sim 540\times$ cheaper, but it was only 7% of the step.

Real target. `compute_fairness_gradients` — 45% of runtime, $\sim 196\text{k}$ needless tensor constructions. Pure-NumPy rewrite next.

All of this is speed only — it is what makes sweeping tree depth affordable.

Tiago Silva · CISUC Data-Centric AI Lab · FADO · 17 July 2026